《高级网页设计 I》

图像解决方案 项目报告

学　　号：24002060419

姓　　名：张志翔

班　　级：计算机 24006

选题名称：虎扑（移动端下导航）

一、实验目的与实验环境

1.1 实验目的

本实验的目标是掌握前端的图像解决方案，重点掌握**字体图标（font-class）**与 SVG 两种矢量图像的获取与使用方式，能够用 CSS 控制图标的尺寸、颜色与排列，并独立完成四级项目；完成后的下导航同时服务于期末三级项目的导航设计。

1.2 实验环境

硬件：PC（Windows 11）。软件：Visual Studio Code 编辑代码；Google Chrome 与 Microsoft Edge 用于多浏览器测试；图标资源来自 iconfont.cn（阿里巴巴矢量图标库）。调试时使用 Chrome DevTools 的设备模拟（Device Toolbar）切换 320 / 375 / 414 / 768 / 1440 五档视口宽度。

1.3 选题说明

本人的三级项目选题为「虎扑页面复刻」，已完成 PC 端的四个页面（index.html、bbs.html、post.html、channel.html）。虎扑 PC 端站点没有底部下导航，因此按照实验指导书“每位同学的项目内容默认为自己期末三级项目选题的下导航，如果所选项目没有下导航，可以选择其他导航或其他移动网站的下导航”的规定，本实验选择与三级项目同源的**虎扑 App 移动端下导航**作为复刻对象。

二、实施过程

2.1 项目需求与实现

（1）原图分析。虎扑 App 的下导航由五个位置组成，依次是“首页 / 专区 / 发布 / 深聊 / 我的”。其中中间的“发布”不是普通 tab，而是一个红色圆形按钮，直径明显大于图标，并且向上凸出导航条；其余四项中，当前选中项使用黑

色实心图标与黑色文字，未选中项使用灰色线性图标与灰色文字；整条导航固定在屏幕最底部，不随内容滚动。原图见图 1。



图 1　虎扑 App 下导航原图（375×812 截屏）



图 2　复刻实现效果（375×812，SVG 版）

对比原图与实现图可以看到，五个位置的数量、顺序、选中态的黑色高亮、中间红色圆形按钮的凸起尺寸都已还原；图标的字形不要求与原图完全一致（指导书明确允许），但保持了“实心房子 / 对话框 / 加号 / 漩涡 / 人形”的语义对应。

（2）图标获取

全部图标均来自 iconfont.cn。在本人自建的图标项目“zhangzhixiang2026”（项目 id 5244713）中共收集了 34 个字形，覆盖本导航所需的全部语义，最终选用 5 个（表 1）。字体包以 woff2 / woff / ttf 三种格式导出，同时得到 iconfont.css。其中“加号”虽然存在于字体包中，但三个加号字形本身都自带一圈圆形外框，放进红色圆钮里会出现“圈套圈”的双环效果，因此发布按钮的加号

改用 CSS 绘制（见 2.2 代码 2）。

导航位置	iconfont 图标名	类名	Unicode	icon_id
首页	首页-选中	icon-index-copy	U+E622	19985212
专区	消息	icon-xiaoxi	U+E64B	1183007
深聊	台风路径	icon-taifenglujing	U+E649	2984471
我的	我的	icon-wode	U+E641	5004078
发布	加号（未采用）	icon-jiahao	U+E60A	3298402

表 1　下导航图标选用对照（图标项目 zhangzhixiang2026，id 5244713）



图 3　图标选用对照与未选中 / 选中两种状态的渲染结果

（3）实现一：SVG 符号雪碧图（方案 A）

index.html、zone.html、chat.html、mine.html 四个页面采用 SVG 方案。做法是在文档顶部放一个隐藏的 <svg>，用 <symbol id="zzx-ico-home"> 这样的定义把 5 个图标的路径存起来，导航里再用 <svg><use href="#zzx-ico-home"></use></svg> 引用。这样 5 个图标只需在页面中出现一次，多处复用等于零成本，且不用额外请求图片文件。

图标颜色通过 fill: currentColor 交给 CSS 的 color 控制，大小通过 width / height 控制，因此同一份图标既能做灰色未选中态，也能做黑色选中态，不需要准备两套图片。

另一个细节是视觉尺寸的统一：不同来源的图标在 1024 的画布里留白不一样，直接同尺寸显示会一大一小。实现时先用 <use> 的 getBBox() 量出每个图标墨迹的实际包围盒，再用 <g transform="translate(x y) scale(k)"> 把墨迹统一缩放到 1024 画布的 920 单位以内，五个图标因此在同一个 23px 的方框里看起来一样大。



图 4　SVG 版下导航特写（放大 4 倍，便于观察图标的实心 / 线性差别）

（4）实现二：字体图标 font-class（方案 B）

按实验要求“使用字体图标或 SVG，要求和上一步中的技术有区别”，方案 A 用了 SVG，方案 B 就必须用字体图标。nav-fontclass.html 复用同一套 CSS 布局，只把图标承载方式换掉：把项目导出的 iconfont.woff2、woff、ttf 放入 font/ 目录，在 css/iconfont.css 中用 @font-face 引入，导航里写 。图标于是变成了一个字符：大小由 font-size 控制，颜色由 color 控制，二者都跟随 CSS 变量。

这里有一个必须处理的问题：iconfont 生成的 .iconfont 基类把 font-size 写成了 16px !important，直接写 .zzx-tabbar__font 无法覆盖。解决办法是用“.zzx-tabbar__font.iconfont”两个类名提高选择器优先级，再用 !important 顶掉基类的 !important，尺寸才能真正由 --zzx-nav-icon 变量统一控制。



图 5　字体图标版下导航（nav-fontclass.html，375×812）

（5）自适应宽度、固定定位与凸起按钮

自适应宽度用弹性盒实现：普通 tab 写 flex: 1 1 0 并配 min-width: 0，四项按可用宽度均分，中间按钮写 flex: 0 0 auto 加固定 48px 宽度，因此无论视口是 320 还是 1440，导航条都不会出现横向滚动或挤压。固定效果用 position: fixed; left/right/bottom: 0 实现，并加 padding-bottom: env(safe-area-inset-bottom, 0px) 适配 iPhone 全面屏的底部安全区。

中间按钮的凸起用 transform: translateY(-8px) 实现，父级刻意不设置 overflow: hidden，否则凸出部分会被裁掉。内容区宽度用 --zzx-page: 750px 限制并居中，页面本体则始终铺满窗口，这样在桌面浏览器下内容是居中的阅读栏，两侧是白底而不是灰边。

（6）两种技术的对比

同一套导航、同一份 CSS 布局，只有图标的承载方式不同。方案 A 把矢量路径内嵌在页面里用〈use〉复用，方案 B 用 @font-face 加载字体文件把图标变成

字符。两者的渲染结果见图 6，代码差异见 2.2。



图 6　SVG 版（方案 A）与字体图标版（方案 B）对比

2.2 项目代码

下导航的核心代码分为四段：字体图标的 HTML 与 CSS、SVG 的 HTML 与 CSS。全部样式表头部与 HTML 头部均注明学号与姓名，类名统一使用 zzx- 前缀（姓名缩写），全站不使用 id 选择器作为样式钩子，全局变量集中在 :root 中定义。

代码 1　字体图标版下导航的 HTML（nav-fontclass.html，节选）

```
<nav class="zzx-tabbar" aria-label="底部导航">
<div class="zzx-tabbar__inner">

<a class="zzx-tabbar__item is-active" href="nav-fontclass.html" aria-current="page">
<i class="iconfont icon-index-copy zzx-tabbar__font" aria-hidden="true"></i>
<span class="zzx-tabbar__text">首页</span>
</a>

<a class="zzx-tabbar__item" href="zone.html">
<i class="iconfont icon-xiaoxi zzx-tabbar__font" aria-hidden="true"></i>
<span class="zzx-tabbar__text">专区</span>
</a>

<!-- 中间凸起的发布按钮，不属于 tab -->
```

```
<a class="zzx-tabbar__publish" href="publish.html" aria-label="发布内容">
<span class="zzx-tabbar__plusmark" aria-hidden="true"></span>
</a>

<a class="zzx-tabbar__item" href="chat.html">
<i class="iconfont icon-taifenglujing zzx-tabbar__font" aria-hidden="true"></i>
<span class="zzx-tabbar__text">深聊</span>
</a>

<a class="zzx-tabbar__item" href="mine.html">
<i class="iconfont icon-wode zzx-tabbar__font" aria-hidden="true"></i>
<span class="zzx-tabbar__text">我的</span>
</a>

</div>
</nav>
```

代码 2　字体图标的引入与尺寸 / 颜色控制（css/iconfont.css、css/hupu-mobile.css，节选）

```
@font-face {
font-family: "iconfont"; /* iconfont 项目 id 5244713 */
src: url('../font/iconfont.woff2?t=1792649896391') format('woff2'),
url('../font/iconfont.woff?t=1792649896391') format('woff'),
url('../font/iconfont.ttf?t=1792649896391') format('truetype');
}
.icon-index-copy:before { content: "\e622"; }
.icon-wode:before { content: "\e641"; }

/* .iconfont 基类把 font-size 写成 16px !important，
用双类选择器提高优先级，让尺寸跟随 CSS 变量 */
.zzx-tabbar__font.iconfont {
font-size: calc(var(--zzx-nav-icon) - 2px) !important;
line-height: 1;
color: inherit; /* 颜色交给 .zzx-tabbar__item 的 color */
}

/* 字体包里只有圈内加号，这里用两条线拼一个纯加号 */
.zzx-tabbar__plusmark::before { width: 18px; height: 2px; }
.zzx-tabbar__plusmark::after { width: 2px; height: 18px; }
```

代码 3　SVG 雪碧图与 <use> 引用（index.html，路径数据略）

```
<!-- 图标雪碧图：只定义一次，页面中不占位 -->
<svg class="zzx-sprite" aria-hidden="true" xmlns="http://www.w3.org/2000/svg">
<symbol id="zzx-ico-home" viewBox="0 0 1024 1024">
<g transform="translate(38.12 36.56) scale(0.92820)">
<path d="M973.9 424.6L578.2 44c-37.9-36.5-97.3-36.5-135.3 0L47.1 424.6 ..." />
</g>
</symbol>
<symbol id="zzx-ico-wode" viewBox="0 0 1024 1024"> ... </symbol>
</svg>

<!-- 导航中引用 -->
```

```
<a class="zzx-tabbar__item is-active" href="index.html" aria-current="page">
<svg class="zzx-tabbar__icon" aria-hidden="true">
<use href="#zzx-ico-home" xlink:href="#zzx-ico-home"></use>
</svg>
<span class="zzx-tabbar__text">首页</span>
</a>
```

代码 4　自适应宽度、固定定位与凸起按钮（css/hupu-mobile.css，节选）

```
:root {
--zzx-nav-h: 52px; /* 导航条高度 */
--zzx-nav-icon: 23px; /* 图标尺寸 */
--zzx-publish: 48px; /* 发布按钮直径 */
--zzx-publish-lift: 8px; /* 向上凸出的距离 */
--zzx-page: 750px; /* 内容区最大宽度 */
}

.zzx-tabbar {
position: fixed; /* 固定在视口底部，随页面滚动不动 */
left: 0; right: 0; bottom: 0;
z-index: 100;
background: #fff;
border-top: 1px solid var(--zzx-line);
padding-bottom: env(safe-area-inset-bottom, 0px); /* 全面屏安全区 */
}

.zzx-tabbar__inner {
display: flex;
align-items: center;
width: 100%;
max-width: var(--zzx-page);
margin: 0 auto;
height: var(--zzx-nav-h);
}

.zzx-tabbar__item {
flex: 1 1 0; /* 四项等分剩余宽度 */
min-width: 0; /* 允许收缩，防止文字撑破 */
height: 100%;
display: flex;
flex-direction: column;
align-items: center;
justify-content: center;
gap: 2px;
color: var(--zzx-sub); /* 未选中：灰色 */
}

.zzx-tabbar__icon {
width: var(--zzx-nav-icon);
height: var(--zzx-nav-icon);
fill: currentColor; /* 颜色跟随 color */
}

.zzx-tabbar__item.is-active { color: var(--zzx-text); } /* 选中：黑色 */
```

```
.zzx-tabbar__publish {
flex: 0 0 auto; /* 不参与等分，固定宽度 */
width: var(--zzx-publish);
height: var(--zzx-publish);
border-radius: 50%;
background: var(--zzx-red-light);
transform: translateY(calc(-1 * var(--zzx-publish-lift)));
}
```

三、测试与发布

3.1 多视口测试

使用 Chrome DevTools 的设备模拟，分别在 320×640（手机 • 小屏）、375×812（手机 • 主流）、414×896（手机 • 大屏）、768×1024（平板）四档视口下检查。四种宽度下五个位置始终等分、文字不换行不溢出、中间按钮的凸起量保持一致，说明 flex: 1 1 0 与 min-width: 0 的配合是有效的。



图 7　下导航在 320 / 375 / 414 / 768 四档视口下的自适应表现

桌面浏览器下（1440×900）页面本体铺满窗口，内容区收在 750px 内并水平居中，导航条同样居中显示，两侧为白底，没有出现灰边或横向滚动条。



图 8　1440×900 桌面浏览器下的表现（内容限宽 750px 居中）

3.2 多浏览器测试

在 Google Chrome 与 Microsoft Edge 两种浏览器中打开同一页面并截图对比。由于两者同为 Chromium 内核，渲染结果一致，下导航的尺寸、圆角、颜色与凸起位置没有差异；两版的截图在 375×812 下逐像素相同，说明页面没有依赖某一浏览器的私有特性。



图 9 Microsoft Edge 下 375×812 视口



图 10 Microsoft Edge 下 1440×900 视口

3.3 页面跳转测试

五个导航项都指向真实页面，不是空链接。点击每一项都能跳转到对应页面，且目标页面上的对应项处于选中态；中间的发布按钮跳转到发布页，该页作为模态页刻意不显示下导航，避免页面上出现两个红色按钮。



图 11　五个页面：每个导航项都有独立页面（发布页为模态页，隐藏导航）

3.4 发布

项目为纯静态页面，通过本地静态服务器发布验证：在 mobile 目录执行 python -m http.server 8899 后，访问 http://127.0.0.1:8899/index.html 即可打开首页。实测六个页面、两个样式表与三个字体文件全部返回 200（其中 font/iconfont.woff2 为 8476 字节），没有任何 404，说明页面不依赖本地文件路径，整体上传到任意静态服务器即可访问。

四、项目总结

本次实验以虎扑 App 的下导航为对象，用 SVG 与字体图标两种技术各实现了一遍。过程中体会最深的有三点。

第一，**矢量图标的关键不在“画得像”，而在“控制得住”。**位图一旦导出，颜色和大小就固定了；而 SVG 与字体图标都能让颜色跟着 CSS 的 color 走、大小跟着 font-size 走，选中态只是把 color 从灰色换成黑色，不需要准备两套图标。这是本次实验最直接的收获。

第二，**两种技术各有代价。**SVG 的优势是所见即所得、可以做多色和复杂路径、不依赖字体文件；代价是图标必须内嵌在页面里，5 个图标就意味着 5 段路径数

据。字体图标的优势是体积极小（woff2 只有 8KB）、引用只需一个类名；代价是只能单色，且必须处理字体加载失败与基类样式覆盖的问题。本次遇到的 .iconfont 基类把 font-size 写成 !important，就是字体图标方案特有的麻烦。

第三，**自适应要落在具体的数值上。**flex: 1 1 0 只是让四项等分，真正不出问题靠的是配套的 min-width: 0（否则文字会撑破容器）和固定宽度的中间按钮（flex: 0 0 auto，不参与等分）。凸起按钮之所以能溢出导航条，前提是父级没有 overflow: hidden；这些条件缺一个，效果就不对。

不足与改进方向：目前图标是单色的，选中态只做颜色与 1px 位移的变化；如果后续要还原虎扑 App 的选中动效，可以把 transform 的过渡时间调长并加入缩放曲线。另外发布页目前只是静态布局，没有接入真正的表单提交。

AI 使用说明：本项目在 AI 工具辅助下完成，使用方式如下。① 用 AI 辅助在 iconfont.cn 上按图标语义检索候选字形，并逐个渲染比对，确定最终选用的 5 个图标；② 由 AI 生成页面骨架与下导航的 CSS，本人对照 App 原图逐项核对位置、尺寸、颜色后再调整；③ 用 AI 辅助完成多视口、多浏览器的截图验证，并检查是否存在横向溢出。所有提交的代码本人均已阅读并理解。